7.Implementation of Random Forest and Performance Analysis

Aim :

To write a program that implements a Random Forest classifier, which combines many decision trees to make predictions, trains it on a training set, predicts the class labels forthe test set, and analyzes its performance using accuracy, the confusion matrix, the classification report and feature importance.

Algorithm :

1. Start the program.
2. Import the required libraries (pandas, matplotlib, train_test_split from sklearn.model_selection,
RandomForestClassifier from sklearn.ensemble, and accuracy_score, confusion_matrix and
classification_report from sklearn.metrics).
3. Load the dataset from the input file.
4. Separate the data into input features (X) and the target variable (y).
5. Split the data into training and testing sets using train_test_split (for example 80% training and
20% testing), with a fixed random_state.
6. Create the RandomForestClassifier model with a chosen number of trees n_estimators (for
example 100) and a fixed random_state.
7. Train the model by fitting it on the training data (X_train, y_train).
8. The model builds many decision trees, each on a random sample of the training data and a
random subset of the features.
9. Predict the class labels for the test set (X_test); each tree votes and the majority class is
chosen as the final prediction.
10. Compare the predicted labels with the actual labels (y_test) and calculate the accuracy.
11. Build the confusion matrix and the classification report (precision, recall and F1-score).
12. Find the importance of each feature using feature_importances_ and display it (optional: plot
it as a bar chart).
13. Display the evaluation results and analyze the performance of the model.
14. Stop the program.

Program,Result&Graph:

